



YACHAY TECH UNIVERSITY

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

PROJECT 1 – TECHNICAL REPORT

Secure Wireless Messaging for IoT Devices

DHE-PSK key exchange and Encrypt-then-MAC on two ESP32 boards

AUTHOR Daniel Troya, Jose Reinoso

COURSE Cryptography

INSTRUCTOR PhD. Christian Iza

URCUQUÍ, ECUADOR

October 2026

Contents

1	Introduction	1
2	System Design	1
2.1	Architecture	1
2.2	Session Key and Device Authentication	2
2.3	Message Protection and Packet Format	3
3	Results	3
3.1	Performance on the ESP32	3
3.2	Security Tests	5
3.3	Discussion	5
4	Conclusions	5
	References	6
A	Additional Material	6
A.1	Screenshots of the Security Tests	6

1 Introduction

Small IoT devices often send data over wireless networks that anyone nearby can listen to, so an attacker can read, change, inject, or repeat packets. The security must therefore rely on the cryptographic protocol and not on the medium, and it must provide confidentiality, integrity, device authentication, and replay protection at the same time, because encryption alone is not secure communication.

Our prototype uses two ESP32 boards running MicroPython, called Alice and Bob, which exchange short text messages over Wi-Fi with UDP. We chose Wi-Fi because the ESP32 supports it natively and it uses standard IP sockets, so the PC can sit between the boards as an attacker with ordinary tools. ESP-NOW was not suitable because its payload is limited to 250 bytes and our handshake messages are larger (274 and 306 bytes), and LoRa would require extra radio modules and has a very low data rate. First, the boards run an ephemeral Diffie–Hellman handshake authenticated with a pre-shared key (PSK) and derive the session keys with HKDF. Then every message is encrypted with AES-256-CBC and protected with an HMAC-SHA256 tag in the Encrypt-then-MAC order, and a session identifier and a sequence number give replay protection. We assume that the attacker controls the network but cannot break AES, SHA-256, or the discrete logarithm problem, and that it does not know the PSK [1].

2 System Design

2.1 Architecture

As figure 1 shows, both boards run the same layered code: `main.py` handles Wi-Fi, the UDP socket, and the menu; `protocol.py` contains the handshake and the `seal` and `open` functions; and the small modules below it call the standard libraries of MicroPython (`cryptolib`, `hashlib`, `hmac`). Only the configuration file changes between Alice (ID 0x01) and Bob (ID 0x02). The PC is used only in the security tests.

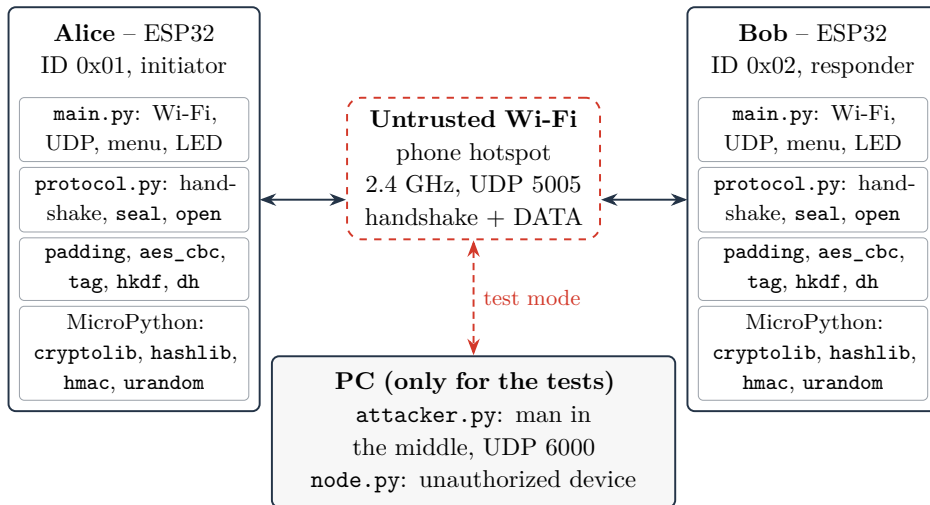


Figure 1. System architecture. The PC is used only in the security tests.

2.2 Session Key and Device Authentication

Plain Diffie–Hellman (DH) gives both parties the same secret $Z = g^{ab} \bmod p$, but it does not tell who they are, so an attacker in the middle could run one exchange with each board. We therefore use an ephemeral DH exchange authenticated with a PSK (DHE-PSK). The exponents are new in every session and erased after use, which gives forward secrecy, and the PSK authenticates the boards. We use the `ffdhe2048` group of RFC 7919 with a 256-bit exponent and check that $1 < Y < p - 1$ [2]. Figure 2 shows the three handshake messages and the first data packet.

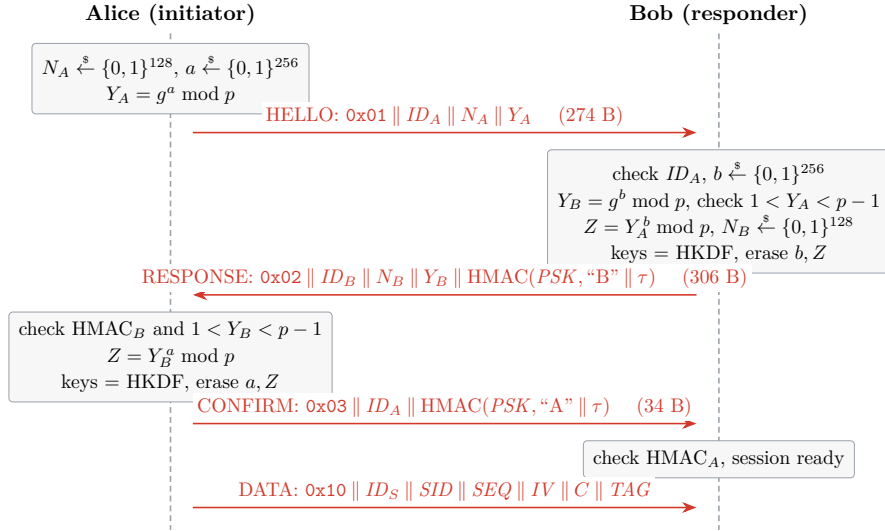


Figure 2. Protocol sequence diagram: DHE-PSK handshake and the first protected message.

Both sides compute the transcript $\tau = ID_A \parallel ID_B \parallel N_A \parallel N_B \parallel Y_A \parallel Y_B$ and derive the session material with HKDF [3]:

$$PRK = \text{HMAC}(N_A \parallel N_B, Z \parallel \text{PSK}), \quad K_{\text{enc}}, K_{\text{mac}}, SID = \text{Expand}(PRK, \text{"enc"}, \text{"mac"}, \text{"sid"}). \quad (1)$$

The nonces are the salt, so two sessions never share keys; K_{enc} and K_{mac} are independent 32-byte keys, and the 4-byte SID is obtained by both boards without being sent. The exponents and Z are erased right after the derivation, the session keys live only in RAM, and the PSK is a random 32-byte value stored only in the configuration file of each board, which is excluded from GitHub.

A device is authorized only if its ID is in the PSK table of the other board and it proves that it knows the PSK: Bob with $\text{HMAC}(\text{PSK}, \text{"B"} \parallel \tau)$ and Alice with $\text{HMAC}(\text{PSK}, \text{"A"} \parallel \tau)$. Since τ covers Y_A and Y_B , an attacker who replaces a public value breaks the check, the labels “A” and “B” prevent reflecting a value back, and the fresh nonces prevent replaying an old handshake. Bob creates the session only after a valid CONFIRM. We used a PSK instead of signatures because MicroPython has no signature library, and writing RSA or ECDSA by hand would be custom cryptography.

2.3 Message Protection and Packet Format

Each message is padded with PKCS#7, encrypted with AES-256-CBC under K_{enc} and a new random IV , and protected with

$$TAG = \text{trunc}_{16}(\text{HMAC}(K_{\text{mac}}, TYPE \parallel ID_S \parallel SID \parallel SEQ \parallel IV \parallel C)). \quad (2)$$

AES-GCM or ChaCha20-Poly1305 would be the usual choice, but `cryptolib` on the ESP32 only offers AES in ECB, CBC, and CTR modes, and writing GCM by hand would be custom cryptography. CBC alone gives no integrity (flipping a bit in a ciphertext block flips the same bit in the next plaintext block without any error), so we use the Encrypt-then-MAC composition with independent keys, which is proven secure [1]. The tag covers the header and C , so any change in the ciphertext, the sender, the session, or the sequence number invalidates it.

Our packet follows the structure $ID_S \parallel SID \parallel SEQ \parallel N \parallel C \parallel TAG$ with two changes: a one-byte $TYPE$ separates handshake and data packets on the same socket, and the nonce N is the CBC IV . Table 1 describes the fields, and a protected packet has $|P| = 42 + 16(\lfloor L/16 \rfloor + 1)$ bytes, where L is the plaintext length.

Table 1. Format of the protected DATA packet.

Field	Size (B)	Purpose
$TYPE$	1	0x10 for data (0x01–0x03 for the handshake).
ID_S	1	Sender; it must be the peer of the session.
SID	4	Current session (derived by HKDF).
SEQ	4	Sequence number, from 1 and increasing; gives freshness.
IV	16	Random IV of AES-CBC (the nonce N).
C	$16(\lfloor L/16 \rfloor + 1)$	Ciphertext of the padded message.
TAG	16	Truncated HMAC-SHA256 of all the previous fields.

The receiver checks a packet in this order: length and type, then SID and sender (a cheap drop), then the tag in constant time, then the sequence number, and only then it decrypts. Verifying the tag before decrypting prevents padding oracle attacks, and checking SEQ only after the tag stops a forged packet with a huge number from blocking the real ones. Replay protection combines the session identifier and the sequence number: inside a session, a packet is accepted only if its SEQ is larger than the last accepted one, and a packet from an old session has another SID and other keys. We did not use timestamps because the boards have no synchronized clock; the cost is that UDP packets arriving out of order are dropped, which is acceptable for short messages sent one at a time.

3 Results

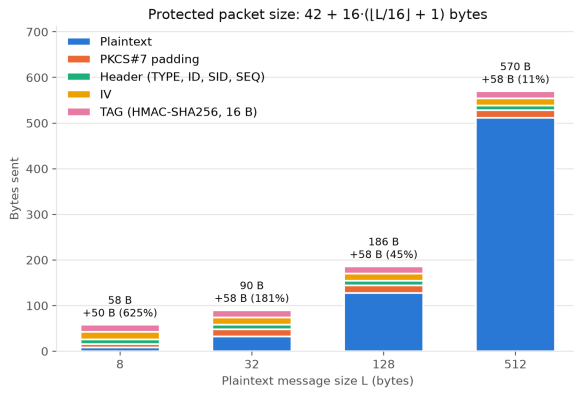
3.1 Performance on the ESP32

All times were measured on the ESP32 with `time.ticks_us()` and are reported as mean $\pm$ standard deviation. The script `medir.py` measures *seal* (padding, encryption, and tag) and

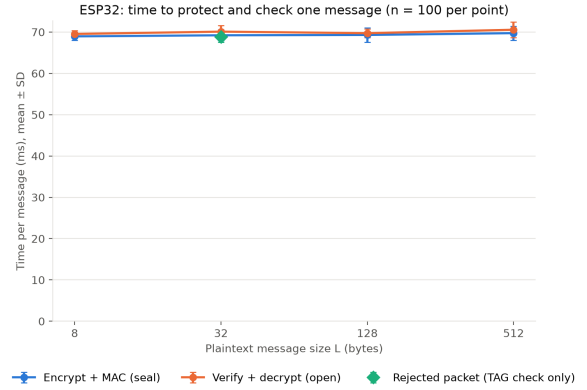
open (checks, decryption, and unpadding) for $L = 8, 32, 128$, and 512 bytes, with 10 warm-up runs and 100 measured runs per size. For the latency, Alice sends 50 protected PING messages and Bob answers each with a protected PONG; the one-way latency is estimated as half of the round-trip time (RTT). Table 2 and figure 3 show the sizes and times.

Table 2. Packet size and time per message on the ESP32 ($n = 100$ per size).

L (B)	Packet (B)	Overhead (B)	Overhead	Seal (ms)	Open (ms)
8	58	50	625%	68.95 ± 0.97	69.54 ± 0.85
32	90	58	181%	69.22 ± 1.34	70.10 ± 1.50
128	186	58	45%	69.30 ± 1.72	69.73 ± 1.04
512	570	58	11%	69.73 ± 1.68	70.56 ± 1.89



(a) Packet size and bytes added.



(b) Seal, open, and reject time.

Figure 3. Cost of protecting one message on the ESP32.

The packet sizes match the formula exactly. Sealing takes about 69 ms and opening about 70 ms, almost independently of L , and rejecting a packet with a bad tag took 68.86 ± 1.30 ms, nearly the same as a full open. The full handshake took 7626.6 ± 23.4 ms ($n = 3$, both roles on one board) and sends 614 bytes once per session. The RTT of a protected PING/PONG was 494.7 ± 87.3 ms ($n = 50$), so the one-way latency is about 247 ms. Each round trip includes two seal and two open operations, about 277 ms, that is, 56% of the RTT (figure 4).

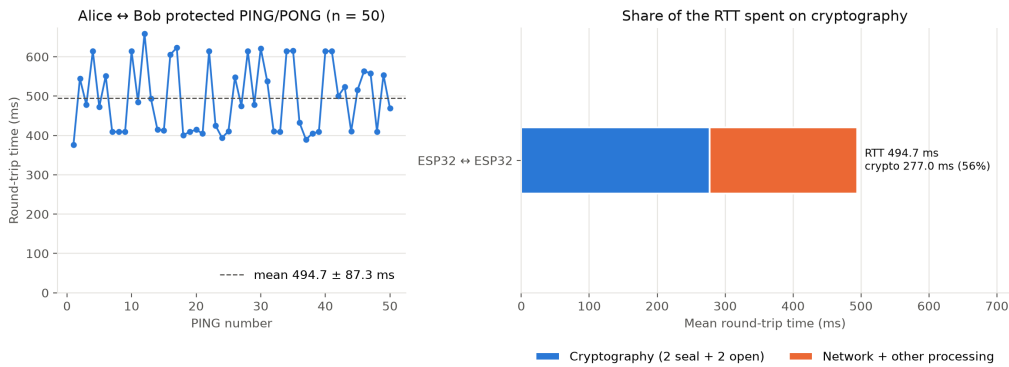


Figure 4. RTT of 50 protected PING/PONG messages between the boards, and the share spent on cryptography.

3.2 Security Tests

The laptop was placed between the boards (`attacker.py`) so that every packet from Alice to Bob went through it, and the unauthorized device was a PC program (`node.py`). Table 3 shows each test and what the receiver printed. In every case the valid message was accepted and shown, and every altered, forged, or repeated message was rejected. All the tests passed; the appendix includes the captures of the most representative ones, and the rest are shown in the live demonstration. The measurements above and these tests are real data but were run in different sessions, so the times printed in the captures (for example, an `open` of about 32 ms) can differ from the table, which is the controlled measurement.

Table 3. Security tests and observed behavior of the receiver.

Test	Observed	Captures
Normal exchange	Bob printed the message; the attacker saw only the header and bytes.	figures 5 and 6
Modified C	One bit of C flipped. Bob: Rejected: TAG.	figures 7 and 8
Modified tag	One bit of the tag flipped. Bob: Rejected: TAG.	live demo
Replay	First copy accepted; second: Rejected: replay.	figures 9 and 10
Forged packet	Random IV , C , and tag. Bob: Rejected: TAG.	figures 11 and 12
MITM	Y_A replaced. Alice: Rejected: handshake.	live demo
Unauthorized device	ID 0x99, and a valid ID with a wrong PSK. Bob: Rejected: handshake; no session.	live demo

3.3 Discussion

The main cost of each message is the HMAC and not AES: rejecting a packet costs almost the same as a full open, and the time hardly changes from 8 to 512 bytes. A likely reason is that the `hmac` module of MicroPython is written in Python, while AES runs in native code. The handshake costs about 110 times more than one message because of its modular exponentiations, but it happens once per session. The protection adds 50 to 58 bytes per message, which is large for very short messages (625% for 8 bytes) but small for long ones (11% for 512 bytes), and every field is needed for one required property. Cryptography takes about 56% of the RTT and the rest is the Wi-Fi network, whose RTT varies a lot (probably because of the power saving of the ESP32 and the phone hotspot), while the cryptographic times vary by only 1 to 2 ms.

The tests agree with the order of the checks. A modified or forged packet fails at the tag, because the HMAC covers the header and the ciphertext and the attacker does not know K_{mac} , so nothing altered is ever decrypted. The replayed packet was different: its tag was valid, so only the sequence number rejected it, which shows that the counter is needed even when the tag is correct.

4 Conclusions

We built a secure messaging system between two ESP32 boards that combines confidentiality, integrity, device authentication, and replay protection in one protocol, using only standard

algorithms and libraries. The session key comes from an ephemeral DH exchange authenticated with a PSK, and, since MicroPython has no AES-GCM, messages use AES-256-CBC with HMAC-SHA256 in the Encrypt-then-MAC order. The tests showed that the receiver accepts only messages from a valid authenticated session. The cost on the ESP32 is about 70 ms and 50 to 58 bytes per message, mostly due to the HMAC, plus a handshake of about 7.6 s once per session.

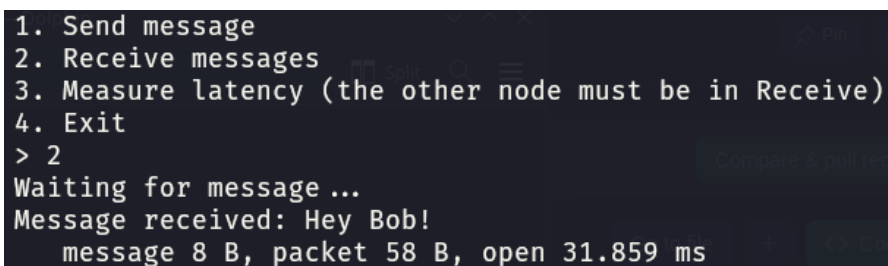
The system does not hide the device IDs, and it cannot stop an attacker from blocking packets or flooding HELLO messages; a stolen PSK would let an attacker impersonate a board in future sessions (past sessions stay secret), and the PSK could be read from the flash with physical access. A sliding window for *SEQ*, a limit on HELLO messages, ECDH, and flash encryption would address these limits.

References

- [1] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*, 3rd ed. Boca Raton, FL, USA: CRC Press, 2020.
- [2] D. K. Gillmor, “Negotiated finite field Diffie-Hellman ephemeral parameters for transport layer security (TLS),” Internet Engineering Task Force, RFC 7919, 2016.
- [3] H. Krawczyk and P. Eronen, “HMAC-based extract-and-expand key derivation function (HKDF),” Internet Engineering Task Force, RFC 5869, 2010.

A Additional Material

A.1 Screenshots of the Security Tests



```
1. Send message
2. Receive messages
3. Measure latency (the other node must be in Receive)
4. Exit
> 2
Waiting for message ...
Message received: Hey Bob!
    message 8 B, packet 58 B, open 31.859 ms
```

Figure 5. Normal exchange: Bob receives the message.


```

Attacker listening on port 6000 | mode: sniff
Node A: ('10.32.7.41', 5005) | Node B: ('10.32.7.224', 5005)
A → B
HELLO 274 B | ID: 0x1
N: 4f46da8cd46cce0b63d85542e47a5b52
Y: 25078c0061007760f72adf76f193f3c3 ...
HELLO forwarded
B → A
RESPONSE 306 B | ID: 0x2
N: 6b7bad06f7d7db155694ba435e78e8d8
Y: b85ccc39ba6a7dcf84fc09872ea54136 ...
RESPONSE forwarded
A → B
CONFIRM 34 B | ID: 0x1
CONFIRM forwarded
A → B
TYPE: 0x10
IDS: 0x1
SID: 30b48fce
SEQ: 1
IV: 4da07b3c7a2c0c6a109ffbb9d9cc6df8
C: 60291be2fb471871fda5206ec2142937
TAG: dffffae3cb9c454f142801c5cbae0b03d
DATA forwarded

```

Figure 6. Sniffing: the attacker only sees bytes.

```

Attacker listening on port 6000 | mode: tamper_c
Node A: ('10.32.7.41', 5005) | Node B: ('10.32.7.224', 5005)
A → B
Original packet received:
TYPE: 0x10
IDS: 0x1
SID: 30b48fce
SEQ: 1
IV: d2dba9f71a5d27d86f781515c7279ef8
C: f922d5c2f26a981a8195c2b1d0f0359b
TAG: efd4a4376a8e2e40619963424247d058
Packet sent (ciphertext modified, 1 bit):
TYPE: 0x10
IDS: 0x1
SID: 30b48fce
SEQ: 2
IV: d2dba9f71a5d27d86f781515c7279ef8
C: f822d5c2f26a981a8195c2b1d0f0359b ← modified
TAG: efd4a4376a8e2e40619963424247d058

```

Figure 7. Modified ciphertext: the attacker changes one bit of C .

```

1. Send message
2. Receive messages
3. Measure latency (the other node must be in Receive)
4. Exit
> 2
Waiting for message ...
Rejected: TAG | packet 58 B

```

Figure 8. Modified ciphertext: Bob rejects the packet (TAG).

```

Attacker listening on port 6000 | mode: replay
Node A: ('10.32.7.41', 5005) | Node B: ('10.32.7.224', 5005)
A → B
Original packet received:
  TYPE: 0x10
  IDS: 0x1
  SID: 30b48fce
  SEQ: 3
  IV: d15ada9bbb66ee2a5f0fc2c0ece53b2f
  C: 93bb231eab929b2d2ee40f8e4ac9d65b
  TAG: c348f3e87ffef9355714fa6970f36f54
Packet sent (1st, normal)
Packet sent again (2nd, the replay, identical to the 1st):
  TYPE: 0x10
  IDS: 0x1
  SID: 30b48fce
  SEQ: 3
  IV: d15ada9bbb66ee2a5f0fc2c0ece53b2f
  C: 93bb231eab929b2d2ee40f8e4ac9d65b
  TAG: c348f3e87ffef9355714fa6970f36f54

```

Figure 9. Replay: the attacker sends the same packet twice.

```

1. Send message
2. Receive messages
3. Measure latency (the other node must be in Receive)
4. Exit
> 2
Waiting for message ...
Message received: Hey Bob
  message 7 B, packet 58 B, open 32.543 ms
Rejected: replay | packet 58 B

```

Figure 10. Replay: Bob accepts the first packet and rejects the second.

```

Attacker listening on port 6000 | mode: forge
Node A: ('10.32.7.41', 5005) | Node B: ('10.32.7.224', 5005)
A → B
Original packet received:
  TYPE: 0x10
  IDS: 0x1
  SID: 30b48fce
  SEQ: 4
  IV: b445f9694484b822a55a351347fae1cb
  C: 8d5a1fb81bc6cd9b958120dcbf54ef1
  TAG: fef761bd7a39de1b05a32f77301ad5b1
Fake packet sent instead of the original:
  TYPE: 0x10
  IDS: 0x1
  SID: 30b48fce
  SEQ: 4
  IV: c3d46ef347fbcd48fd701555c74b2f2 ← modified
  C: fd2c8b97f2f74363cf1f60c6f4a824b7 ← modified
  TAG: e7b5d89b0bda0f68166b09949e5cfbfd ← modified

```

Figure 11. Forgery: the attacker replaces IV, C, and TAG.

```
1. Send message
2. Receive messages
3. Measure latency (the other node must be in Receive)
4. Exit
> 2
Waiting for message ...
Rejected: TAG | packet 58 B
```

Figure 12. Forgery: Bob rejects the packet (TAG).